

BLM3522 Bulut Bilişim ve Uygulamaları Dersi

PROJE 4 :

IOT Ve Akıllı Şehir Uygulaması

**Yağmur Kalyoncu
23291413**



github linki: <https://github.com/Yagmurkalyoncu/iot-akilli-sehir>



video linki : <https://www.youtube.com/watch?v=vxwVIW6NFsY>

İçindekiler

1. Giriş ve Proje Amacı	3
2. Kullanılan Teknolojiler	3
2.1 Programlama Dili ve Kütüphaneler	3
2.2 Bulut Platformu (AWS) ve IoT Protokolleri	3
3. Sistem Mimarisi	4
4. IoT Sensör Simülatörü	4
4.1 Sensör Listesi	4
5. AWS IoT Core Entegrasyonu	5
5.1 Thing Oluşturma	5
5.2 Sertifika ve Güvenlik	5
5.3 MQTT Bağlantısı	5
6.AWS DynamoDB Entegrasyonu	6
7.Veri Görselleştirme	7
8.Sonuç ve Değerlendirme.....	7

1. Giriş ve Proje Amacı

Bu proje kapsamında; yerel donanım bağımlılıklarını ortadan kaldıran, akıllı şehir bileşenlerini (trafik yoğunluğu, çevre sıcaklığı, gürültü kirliliği ve akıllı aydınlatma) simüle eden otonom bir veri hattı (data pipeline) kurgulanmıştır.

- Projenin temel amacı;** IoT dünyasının endüstriyel standardı olan hafif ve olay güdümlü (event-driven) MQTT protokolünün AWS IoT Core mimarisi üzerindeki entegrasyon süreçlerini uygulamalı olarak analiz etmektir.
- Python ile geliştirilen sensör simülatörünün ürettiği veriler, MQTT protokolü (Port 8883) ve X.509 dijital sertifika tabanlı kriptografik güvenlik mekanizmaları kullanılarak AWS IoT Core'a iletilmekte, AWS DynamoDB NoSQL veritabanında saklanmakta ve Matplotlib ile görselleştirilmektedir.

2. Kullanılan Teknolojiler ve Sistem Bileşenleri

2.1 Programlama Dili ve Kütüphaneler

- Python 3.11:** IoT ajan betiklerinin çalıştırılması ve veri manipülasyonu süreçleri için ana backend dili olarak kullanılmıştır.
- AWSIoTPythonSDK:** Cihazların AWS IoT Core MQTT Broker altyapısına asenkron, çift yönlü ve güvenli bir şekilde bağlanmasını sağlayan temel SDK paketidir.
- Boto3 (AWS SDK for Python):** Yerel veya sunucu tabanlı betiklerin AWS DynamoDB NoSQL tablolarıyla doğrudan API entegrasyonunu yürütmek amacıyla tercih edilmiştir.
- Matplotlib:** IoT sensörlerinden toplanan zaman serisi (time-series) analitik metaverilerinin 4 metrikli entegre bir dashboard üzerinde görselleştirilmesi için kullanılmıştır.

2.2 Bulut Platformu (AWS) ve IoT Protokolleri

Teknoloji / Servis	Projedeki Fonksiyonel Amacı
AWS IoT Core	Cihaz kaydı, dijital gölge (Thing), mesaj yönlendirme ve MQTT broker yönetimi.
MQTT	Hafif, düşük paket boyutuna sahip ve asenkron veri iletim protokolü.
Amazon DynamoDB	Akıllı şehir sensörlerinden akan telemetri verilerinin kalıcı olarak saklanması.
AWS IAM / IoT Policy	Cihazların hangi topic'lere yazıp okuyabileceğini belirleyen yetkilendirme katmanı.

3. Sistem Mimarisi

Veri akış adımları sırasıyla şu şekildedir:

1. Sensör Simülatörü — Python ile 5 farklı IoT sensörü simüle edilmektedir.
2. MQTT Bağlantısı — AWSIoTPythonSDK ile TLS/SSL güvenli bağlantı kurulur.
3. AWS IoT Core — Mesajlar 'akilli-sehir/sensor-verisi' topic'ine yayınlanır.
4. DynamoDB Entegrasyonu — Her sensör ölçümü UUID ile Amazon DynamoDB NoSQL veritabanına kalıcı olarak kaydedilir.
5. Görselleştirme — Toplanan veriler, Matplotlib kütüphanesi kullanılarak 4 farklı metrik grafikte analiz edilir.

4. IoT Sensör Simülatörü

4.1 Sensör Listesi

Sensör ID	Konum	Tip	Ölçüm Parametreleri
trafik-001	Atatürk Bulvarı	Trafik	Araç sayısı, yoğunluk, hız
trafik-002	Cumhuriyet Caddesi	Trafik	Araç sayısı, yoğunluk, hız
cevre-001	Merkez Park	Çevre	Sıcaklık, nem, gürültü, hava kalitesi
cevre-002	Sanayi Bölgesi	Çevre	Sıcaklık, nem, gürültü, hava kalitesi
lambda-001	Ana Cadde	Sokak Lambası	Durum, enerji tüketimi, parlaklık

```
def veri_uret(sensor):
    if sensor["tip"] == "trafik":
        olcum = trafik_verisi()
    elif sensor["tip"] == "cevre":
        olcum = cevre_verisi()
    else:
        olcum = lambda_verisi()

    return {
        "sensor_id": sensor["id"],
        "konum": sensor["konum"],
        "tip": sensor["tip"],
        "olcum": olcum,
        "zaman": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    }
```

```
{
  "sensor_id": "cevre-002",
  "konum": "Sanayi Bölgesi",
  "tip": "cevre",
  "olcum": {
    "sicaklik_c": 34.8,
    "nem_yuzde": 23.3,
    "gurultu_db": 84.6,
    "hava_kalitesi": 74
  },
  "zaman": "2026-05-26 17:18:27"
}

{
  "sensor_id": "lambda-001",
  "konum": "Ana Cadde",
  "tip": "sokak_lambasi",
  "olcum": {
    "durum": "Acik",
    "enerji_tuketim_w": 105,
    "parlaklik_yuzde": 54
  },
  "zaman": "2026-05-26 17:18:27"
}
```

Toplam 5 sensörden veri toplandı!
PS C:\Users\HP\Desktop\iot-akilli-sehir>

- Simülatör tarafından üretilen veri paketi ham string formatında değil, bulut servislerinin doğrudan parse edebileceği, standartlara uygun asenkron JSON formatında yapılandırılmıştır

5. AWS IoT Core Entegrasyonu

5.1 Thing Oluřturma

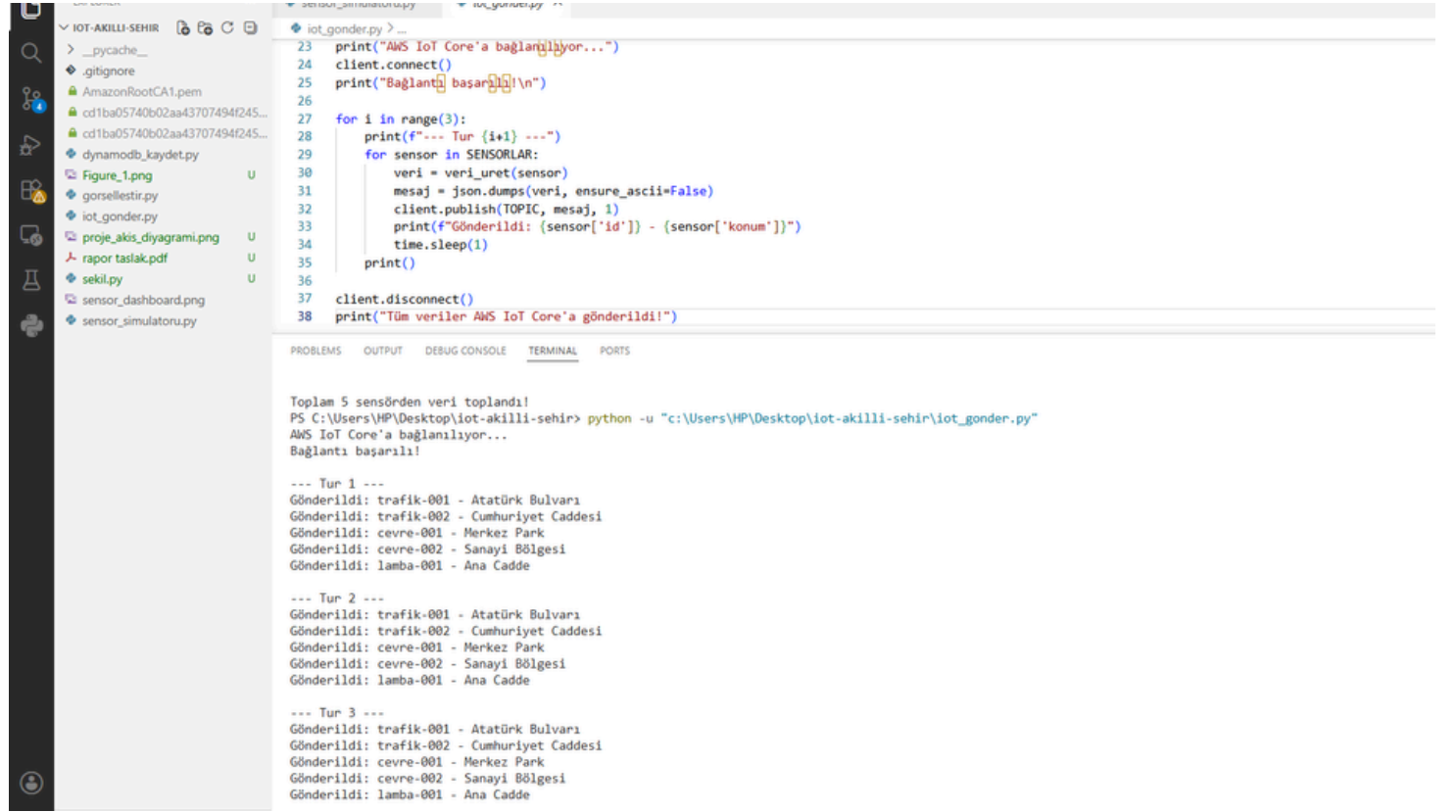
AWS IoT Core'da 'akilli-sehir-sensor' adlı bir Thing (cihaz) oluřturulmuřtur. Bu Thing, fiziksel veya sanal IoT cihazlarının buluttaki dijital temsilini oluřturmaktadır.

5.2 Sertifika ve Gvenlik

AWS IoT Core, cihaz kimlik doęrulama iin X.509 sertifikaları kullanmaktadır. 'iot- sensor-policy' adlı IAM politikası ile Connect, Publish, Subscribe ve Receive izinleri verilmiřtir. Baęlantı TLS 1.2 řifrelemesiyle gvence altına alınmıřtır.

5.3 MQTT Baęlantısı

MQTT protokol port 8883 zerinden kullanılmıřtır. Sensr verileri 'akilli-sehir/sensor- verisi' topic'ine yayınlanmıřtır. Her 3 turda toplam 15 mesaj bařarıyla iletilmiřtir.



```
23 print("AWS IoT Core'a baęlanılıyor...")
24 client.connect()
25 print("Baęlantı bařarıldı!\n")
26
27 for i in range(3):
28     print(f"--- Tur {i+1} ---")
29     for sensor in SENSORLAR:
30         veri = veri_uret(sensor)
31         mesaj = json.dumps(veri, ensure_ascii=False)
32         client.publish(TOPIC, mesaj, 1)
33         print(f"Gnderildi: {sensor['id']} - {sensor['konum']}")
34         time.sleep(1)
35     print()
36
37 client.disconnect()
38 print("Tm veriler AWS IoT Core'a gnderildi!")
```

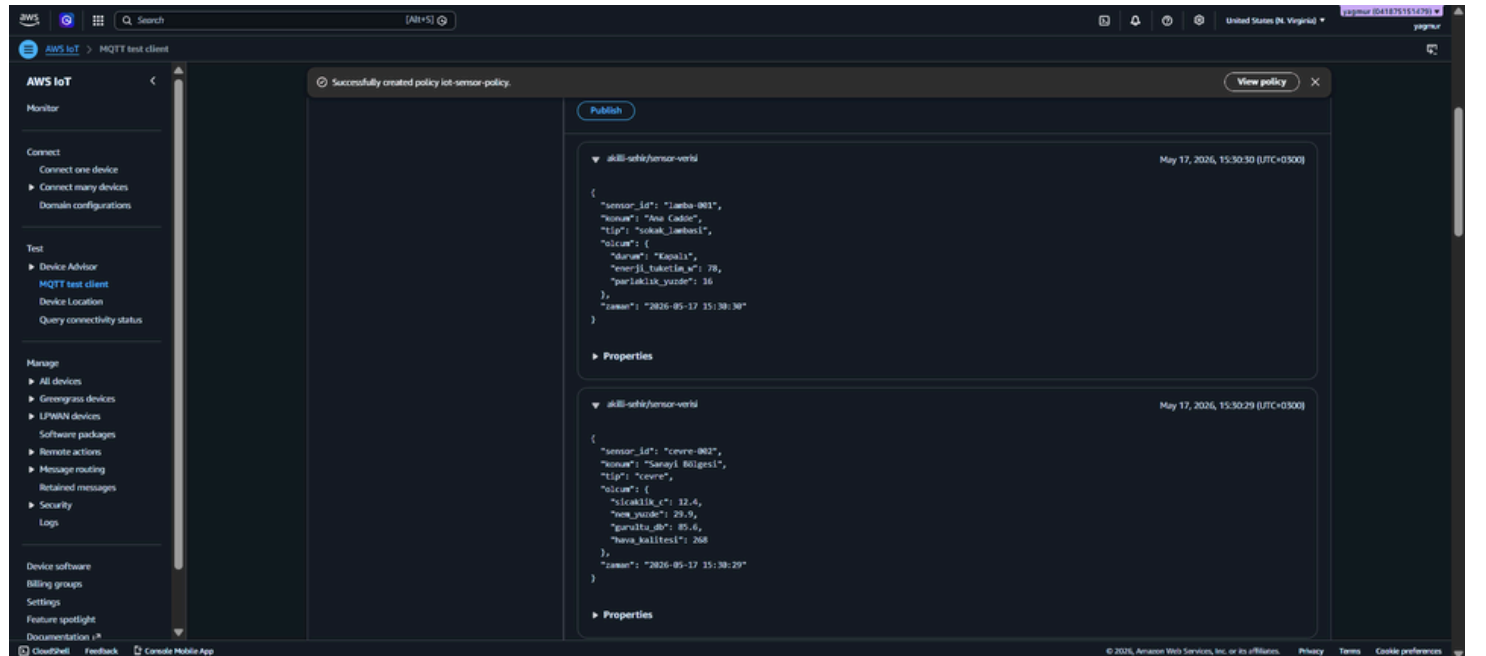
Problems OUTPUT DEBUG CONSOLE TERMINAL PORTS

Toplam 5 sensrden veri toplandı!
PS C:\Users\HP\Desktop\iot-akilli-sehir> python -u "c:\Users\HP\Desktop\iot-akilli-sehir\iot_gonder.py"
AWS IoT Core'a baęlanılıyor...
Baęlantı bařarıldı!

--- Tur 1 ---
Gnderildi: trafik-001 - Atatrk Bulvarı
Gnderildi: trafik-002 - Cumhuriyet Caddesi
Gnderildi: cevne-001 - Merkez Park
Gnderildi: cevne-002 - Sanayi Blgesi
Gnderildi: lamba-001 - Ana Cadde

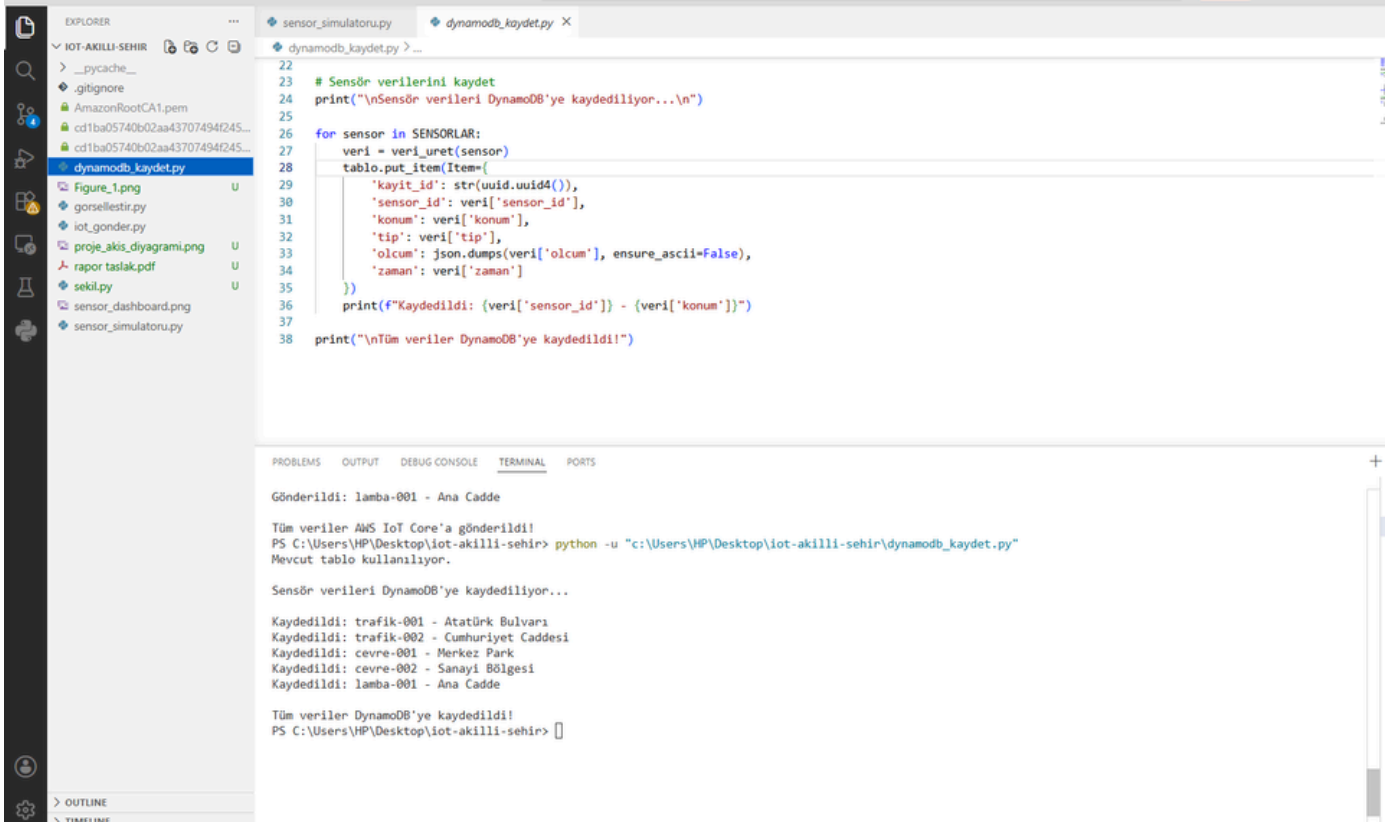
--- Tur 2 ---
Gnderildi: trafik-001 - Atatrk Bulvarı
Gnderildi: trafik-002 - Cumhuriyet Caddesi
Gnderildi: cevne-001 - Merkez Park
Gnderildi: cevne-002 - Sanayi Blgesi
Gnderildi: lamba-001 - Ana Cadde

--- Tur 3 ---
Gnderildi: trafik-001 - Atatrk Bulvarı
Gnderildi: trafik-002 - Cumhuriyet Caddesi
Gnderildi: cevne-001 - Merkez Park
Gnderildi: cevne-002 - Sanayi Blgesi
Gnderildi: lamba-001 - Ana Cadde



6. AWS DynamoDB Entegrasyonu

Sensör verileri AWS DynamoDB 'iot-sensor-verileri' tablosuna kaydedilmiştir.



```
22
23 # Sensör verilerini kaydet
24 print("\nSensör verileri DynamoDB'ye kaydediliyor...\n")
25
26 for sensor in SENSORLAR:
27     veri = veriyetir(sensor)
28     tablo.put_item(Item={
29         'kayit_id': str(uuid.uuid4()),
30         'sensor_id': veri['sensor_id'],
31         'konum': veri['konum'],
32         'tip': veri['tip'],
33         'olcum': json.dumps(veri['olcum'], ensure_ascii=False),
34         'zaman': veri['zaman']
35     })
36     print(f"Kaydedildi: {veri['sensor_id']} - {veri['konum']}")
37
38 print("\nTüm veriler DynamoDB'ye kaydedildi!")
```

Gönderildi: lamba-001 - Ana Cadde

Tüm veriler AWS IoT Core'a gönderildi!

PS C:\Users\HP\Desktop\iot-akilli-sehir> python -u "c:\Users\HP\Desktop\iot-akilli-sehir\dynamodb_kaydet.py"

Mevcut tablo kullanılıyor.

Sensör verileri DynamoDB'ye kaydediliyor...

Kaydedildi: trafik-001 - Atatürk Bulvarı

Kaydedildi: trafik-002 - Cumhuriyet Caddesi

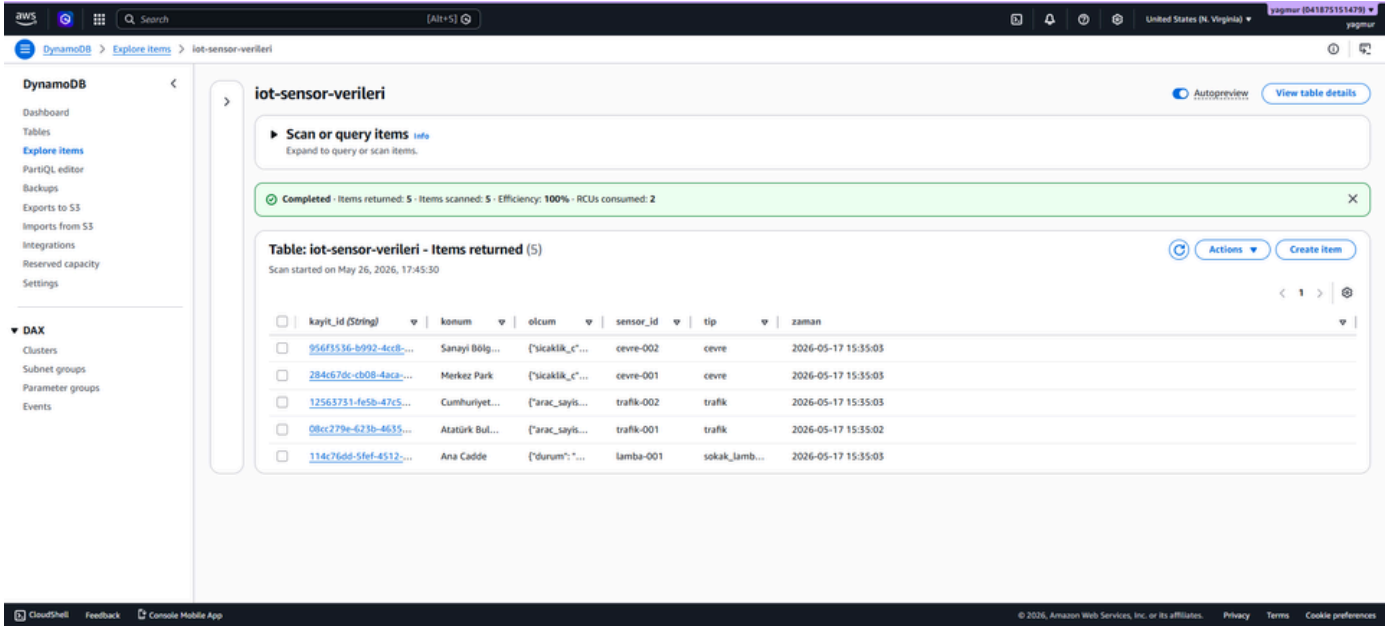
Kaydedildi: cevre-001 - Merkez Park

Kaydedildi: cevre-002 - Sanayi Bölgesi

Kaydedildi: lamba-001 - Ana Cadde

Tüm veriler DynamoDB'ye kaydedildi!

PS C:\Users\HP\Desktop\iot-akilli-sehir>



	kayit_id (String)	konum	olcum	sensor_id	tip	zaman
☐	956f5536-b992-4cc8-...	Sanayi Bölgesi	{ "sicaklik_c": ...	cevre-002	cevre	2026-05-17 15:35:03
☐	284c67dc-cb08-4aca-...	Merkez Park	{ "sicaklik_c": ...	cevre-001	cevre	2026-05-17 15:35:03
☐	12563731-fe5b-47c5-...	Cumhuriyet Caddesi	{ "arac_sayisi": ...	trafik-002	trafik	2026-05-17 15:35:03
☐	08cc279e-623b-4635-...	Atatürk Bulvarı	{ "arac_sayisi": ...	trafik-001	trafik	2026-05-17 15:35:02
☐	114c76dd-5fef-4512-...	Ana Cadde	{ "durum": "..."	lamba-001	sokak_lamb...	2026-05-17 15:35:03

Tablo şeması şu alanları barındırmaktadır:

- **kayit_id (Partition Key - String):** Dağıtık IoT ağlarında veri çakışmalarını önlemek amacıyla Python katmanında UUID algoritması ile üretilen evrensel benzersiz anahtardır.
- **sensor_id (String):** Ölçümü yapan cihaza ait benzersiz kimlik referansıdır.
- **konum & tip (String):** Sensörün coğrafi lokasyonu ve ölçüm kategorisidir.
- **olcum (Map / JSON):** İlgili sensörün anlık ürettiği dinamik ölçüm değerleridir.
- **zaman (String):** Zaman serisi analizleri için ISO 8601 standardında üretilen tarih damgasıdır

7. Veri Görselleştirme

Toplanan 10 ardışık ölçüm döngüsüne ait telemetri verileri, kentsel analiz uzmanlarının kararlarını kolaylaştırmak amacıyla Matplotlib motoru ile görselleştirilmiştir.

1. Trafik Yoğunluğu (Atatürk Bulvarı): Zaman serisi çizgi grafiği olup, alt alanı doldurularak trend analizi belirginleştirilmiştir. %70 kritik yoğunluk Eşiği takip edilmektedir.

2.Sıcaklık Değişimi (Merkez Park): 5°C ile 40°C arasında dalgalanan mikro-klimatik hava değişimlerini kare sembollü trend çizgisiyle sunar.

3.Gürültü Seviyesi (Merkez Park): Akustik kirliliği izleyen sütun grafiğidir. Grafiğe enjekte edilen 65 dB Kentsel Gürültü Limit Çizgisi, eşik aşımalarını anlık gösteren bir karar destek mekanizmasıdır.

4.Sokak Lambası Enerji Tüketimi: Akıllı aydınlatma düğümünün watt cinsinden anlık yük durumunu dinamik sütunlarla raporlar.



8. Sonuç ve Değerlendirme

Bu proje kapsamında Python ve AWSbulut servisleri kullanılarak eksiksiz bir IoT ve Akıllı Şehir uygulaması başarıyla geliştirilmiştir. 5 farklı sensörden toplanan veriler MQTT protokolü ile AWS IoT Core'a iletilmiş, DynamoDB'de saklanmış ve görselleştirilmiştir.

AWS IoT Core'un MQTT broker özelliği sayesinde sensör verileri gerçek zamanlı olarak buluta aktarılmıştır. MQTT test client'ta mesajların anlık geldiği doğrulanmıştır.